INSTITUT TEKNOLOGI DEL

FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO

PROGRAM STUDI S-1 INFORMATIKA

PROPOSAL PENGEMBANGAN PROGRAM APLIKASI WEB SEDERHANA

TUGAS I: 11S3133 – PENGEMBANGAN APLIKASI BERBASIS WEB

"Pengembangan Aplikasi Prakiraan Cuaca Real-Time (Weather Web App) Berbasis Vanilla Web Technologies dan Integrasi Open-Meteo REST API"

Nama Mahasiswa	: **Yogi Tri Medya Panjaitan**
NIM	: **11S24023**
Program Studi	: S-1 Informatika
Mata Kuliah	: 11S3133 – Pengembangan Aplikasi Berbasis Web (PABWE)
Tahun Akademik	: 2026/2027 – Semester Genap
Batas Linimasa	:1 (Satu) Minggu (Penugasan Individu)

Repository: https://github.com/yogitm/weather-web-app.git

LAGUBOTI, TOBA, SUMATERA UTARA
OKTOBER 2026

DAFTAR ISI

1. Halaman Judul dan Informasi Dokumen
2. BAB I: Pendahuluan
    - 1.1 Latar Belakang dan Transformasi Generative AI dalam SDLC Web Frontend
    - 1.2 Rumusan Masalah dan Urgensi Pengembangan Weather Web App
    - 1.3 Tujuan dan Manfaat Pengembangan
    - 1.4 Ruang Lingkup dan Batasan Aplikasi (Linimasa 1 Minggu)
3. BAB II: Spesifikasi dan Perancangan Arsitektur Perangkat Lunak
    - 2.1 Pemilihan Tumpukan Teknologi (Technology Stack)
    - 2.2 Arsitektur Sistem Modular Berbasis Lapisan (Layered Clean Architecture)
    - 2.3 Alur Kerja Integrasi Multi-Endpoint REST API (Chained Asynchronous Flow)
    - 2.4 Spesifikasi Kebutuhan Fungsional dan Non-Fungsional
4. BAB III: Proses Interaksi dan Rekayasa Prompt dengan AI Generatif
    - 3.1 Skenario Rekayasa Kebutuhan dan Prompt Engineering
    - 3.2 Iterasi Prompt I: Perancangan Semantik HTML5 Aksesibel dan Sistem Desain Glassmorphism CSS3
    - 3.3 Iterasi Prompt II: Pembangunan Lapisan Layanan API (WeatherService) dengan Chained Fetch

    3.4 Iterasi Prompt III: Orkestrasi Lapisan Antarmuka (UIController), Penanganan Asinkron, dan Pemetaan WMO
5. BAB IV: Evaluasi Kritis dan Verifikasi Teknis Mandiri
    - 4.1 Identifikasi Anomali dan Cacat Logika pada Kode Bawaan AI
    - 4.2 Analisis Masalah I: Karakteristik Fetch API (Ketiadaan Reject Otomatis pada HTTP 404/500)
    - 4.3 Analisis Masalah II: Kerentanan Keamanan XSS (Cross-Site Scripting) pada Manipulasi DOM
    - 4.4 Analisis Masalah III: Jebakan Asinkron (Stuck Loading State Akibat Ketiadaan Blok Finally)
    - 4.5 Analisis Masalah IV: Ketidaklengkapan Pemetaan Standar Meteorologi (WMO Weather Codes)
    - 4.6 Perbandingan Kode Komparatif: Sebelum (AI-Generated) vs. Sesudah Refactoring (HumanVerified)
6. BAB V: Strategi Pengelolaan Repositori dan Kontrol Versi
    - 5.1 Struktur Repositori Terorganisir dan Standar Berkas Proyek
    - 5.2 Standar Konvensi Commit (Conventional Commits 1.0.0)
    - 5.3 Kebijakan Transparansi, Lisensi, dan Aksesibilitas Dosen/Asisten
7. BAB VI: Rencana Implementasi dan Linimasa Kerja (1 Minggu)
8. BAB VII: Kesimpulan dan Refleksi Kritis Mahasiswa
9. Daftar Pustaka

BAB I: PENDAHULUAN

1.1 Latar Belakang dan Transformasi Generative AI dalam SDLC Web Frontend

Perkembangan teknologi kecerdasan buatan generatif (*Generative Artificial Intelligence* / Gen-AI) yang digerakkan oleh model bahasa berskala besar (*Large Language Models* - LLM) seperti OpenAI GPT, Anthropic Claude, dan Google Gemini telah mentransformasi metodologi rekayasa perangkat lunak modern (*Software Development Life Cycle* - SDLC). Pada domain pengembangan aplikasi web berbasis *client-side* (frontend), Generative AI telah menggeser pola kerja konvensional menjadi sebuah kolaborasi berkecepatan tinggi:

1. **Perancangan Antarmuka dan Tata Letak (*Layout & UI Design*)**: AI memfasilitasi percepatan sintesis struktur semantik HTML5, pemetaan pohon hierarki elemen dokumen (*Document Object Model* DOM), serta penyusunan sistem token visual CSS3 modern (seperti *CSS Custom Properties*, Flexbox, CSS Grid, dan efek visual *glassmorphism*).
2. **Penulisan Kode Program Logika Bisnis (*Code Authoring*)**: AI mampu menyusun algoritma pemrosesan data, pemetaan struktur JSON kompleks, integrasi *asynchronous HTTP requests* menggunakan *Fetch API*, hingga pemisahan modul berbasis fungsi tanggung jawab tunggal (*Single Responsibility Principle*).
3. **Pengujian Kode (*Automated & Manual Testing*)**: Gen-AI membantu menyusun skenario pengujian variasi input tepi (*edge-case testing*), seperti penanganan respon HTTP non-200, input string kosong, karakter khusus yang rentan terhadap injeksi, dan latensi jaringan tinggi.
4. **Debugging dan Refactoring**: Kemampuan LLM dalam membedah pesan kesalahan pada konsol (*browser console stack trace*) membantu pengembang melacak kegagalan *asynchronous promise*, inkonsistensi tipe data, atau kebocoran memori pada *event listener*.

Kendati demikian, implementasi Generative AI dalam pengembangan web murni (*Vanilla Web Technologies*) sering kali melahirkan masalah kritis: model AI cenderung memproduksi kode yang tampak berjalan secara visual (*happy path*), namun mengabaikan penanganan kesalahan jaringan yang mendalam, rentan terhadap celah keamanan injeksi skrip lintas situs (*Cross-Site Scripting* - XSS), serta tidak mematuhi standar aksesibilitas web (*Web Content Accessibility Guidelines* - WCAG). Oleh karena itu, pengembang manusia tetap memegang peranan krusial sebagai auditor teknis dan pengambil keputusan rekayasa yang bertanggung jawab penuh atas keandalan aplikasi.

1.2 Rumusan Masalah dan Urgensi Pengembangan Weather Web App

Dalam kurikulum mata kuliah **11S3133 - Pengembangan Aplikasi Berbasis Web (PABWE)** di Institut Teknologi Del, penguasaan terhadap teknologi web fundamental—yakni HTML5 semantik, CSS3 murni berstandar modern, dan Vanilla JavaScript (ES6+)—merupakan kompetensi prasyarat mutlak sebelum mahasiswa melangkah ke framework tingkat lanjut.

Aplikasi prakiraan cuaca real-time (**Weather Web App**) dipilih sebagai studi kasus pengembangan Tugas I karena merepresentasikan tantangan komputasi web modern yang nyata:

- Kebutuhan integrasi data dinamis dari pihak ketiga melalui RESTful API publik (*Open-Meteo API*).
- Kebutuhan orkestrasi asinkron berantai (*chained asynchronous requests*): mengonversi nama kota menjadi koordinat geografis via *Geocoding API*, lalu mengambil data meteorologis via *Forecast API*.

- Pengelolaan status antarmuka pengguna (*feedback states*) yang tangguh: status memuat data (*loading spinner*), status galat (*error banner* ramah pengguna), dan status penyajian data cuaca (*weather card display*).
- Desain antarmuka yang estetis, responsif, berkinerja tinggi (*zero-dependency*), dan mematuhi kaidah aksesibilitas (A11y).

Tantangan utama yang diangkat dalam proposal ini adalah bagaimana merancang, membangun, dan menyempurnakan aplikasi web cuaca ini secara mandiri dalam linimasa 1 (satu) minggu dengan memanfaatkan Gen-AI sebagai asisten produktivitas, sembari mendemonstrasikan verifikasi teknis kritis terhadap cacat kode yang dihasilkan AI.

1.3 Tujuan dan Manfaat Pengembangan

Tujuan dari pelaksanaan proyek Tugas I ini adalah:

1. Mengembangkan aplikasi web prakiraan cuaca real-time yang mandiri tanpa dependensi pustaka eksternal (*pure vanilla stack*: HTML5, CSS3, ES6+ JavaScript).
2. Mengintegrasikan layanan *Open-Meteo Geocoding API* dan *Open-Meteo Weather Forecast API* secara asinkron menggunakan paradigma `async/await` .
3. Menerapkan arsitektur kode modular berorientasi objek yang bersih (*Clean Separation of Concerns*), memisahkan lapisan layanan data (`WeatherService`) dengan lapisan manipulasi antarmuka (`UIController`).
4. Mendokumentasikan proses rekayasa instruksi (*Prompt Engineering*) berjenjang kepada AI Generatif, disertai rasionalisasi teknis di balik setiap prompt.
5. Melakukan audit kritis, menemukan anomali logika bawaan AI, dan mengeksekusi *refactoring* mandiri untuk menjamin ketahanan sistem terhadap galat (*fault-tolerance*) dan keamanan web.
6. Mempublikasikan seluruh artefak kode dan dokumentasi pada repositori publik GitHub dengan riwayat komitmen yang rapi untuk dievaluasi oleh Dosen dan Tim Asisten Dosen.

Manfaat proyek ini adalah memberikan tolok ukur kompetensi mahasiswa S-1 Informatika dalam menguasai teknologi dasar web, kemampuan integrasi API terbuka, serta kematangan berpikir kritis dalam memvalidasi keluaran kecerdasan buatan.

1.4 Ruang Lingkup dan Batasan Aplikasi (Linimasa 1 Minggu)

Agar pengembangan dapat diselesaikan tepat waktu dalam linimasa 7 hari kerja individu, batasan proyek ditetapkan sebagai berikut:

- **Teknologi Utama**: Pure Vanilla HTML5, CSS3, dan Vanilla JavaScript (ES6+) tanpa framework CSS (Bootstrap/Tailwind) maupun framework JS (React/Vue/Angular).
- **Penyedia Data Cuaca**: Open-Meteo API (Layanan REST API cuaca terbuka tanpa keharusan konfigurasi API Key rahasia di sisi klien) yang dipadukan dengan repositori aset ikon resmi OpenWeatherMap.
- **Fitur Utama**:
    1. Pencarian cuaca real-time berdasarkan input nama kota di seluruh dunia.

2. Pemuatan otomatis data cuaca kota *default* (Jakarta) saat pertama kali halaman dimuat (*auto-load default city*).
3. Visualisasi data cuaca utama: nama kota & negara, waktu pembaruan lokal, ikon grafis cuaca teranimasi, nilai suhu (°C), dan deskripsi berbahasa Indonesia.
4. Visualisasi metrik sekunder: persentase kelembapan udara (`%`) dan kecepatan angin (`m/s`).
5. Penanganan galat interaktif (indikator *spinner* murni CSS saat memuat, serta banner peringatan informatif saat kota tidak ditemukan atau jaringan offline).

- **Batasan**: Aplikasi berfokus pada kondisi cuaca *current real-time* (tidak mencakup prediksi historis multi-dekade atau grafik radar cuaca Doppler interaktif).

BAB II: SPESIFIKASI DAN PERANCANGAN ARSITEKTUR PERANGKAT LUNAK

2.1 Pemilihan Tumpukan Teknologi (*Technology Stack*)

Keputusan perancangan arsitektur didasarkan pada prinsip efisiensi, performa tinggi, dan *zero build-step* (dapat langsung dieksekusi pada peramban web modern tanpa proses kompilasi bundler):

- **Struktur Dokumen**: **HTML5 Semantik** (`<main>` , `<header>` , `<form role="search">` , `<section>` , `<article>` , `<footer>`) dengan atribut aksesibilitas WAI-ARIA (`aria-live="polite"` , `aria-label` , `aria-hidden`).
- **Tata Letak & Gaya**: **CSS3 Murni** dengan variabel desain (*CSS Custom Properties*), tata letak Flexbox dan CSS Grid 2 kolom, teknik *backdrop-filter blur* (Glassmorphism), animasi `@keyframes` , serta desain *mobile-first* (`@media (max-width: 480px)`).
- **Tipografi**: Google Fonts (*Plus Jakarta Sans*) dengan konfigurasi *preconnect* untuk optimalisasi waktu muat (*render-blocking minimization*).
- **Logika & Integrasi**: **Vanilla JavaScript (ECMAScript 2020+)** dengan mode ketat (`'use strict'`), sintaks `async/await` , `fetch()` , *Object Destructuring*, dan manipulasi DOM berbasis referensi cache.
- **Penyedia Data**: Open-Meteo Geocoding API (`geocoding-api.open-meteo.com`) & Open-Meteo Weather Forecast API (`api.open-meteo.com`).



```
---+                                          | HTTP Asynchronous Fetch
v
+-------------------------------------------------------------------------------+
|                              REST API CLOUD SERVICES                          |
|                                                                               |
|   1. Open-Meteo Geocoding API:  /v1/search?name={city}&count=1                |
|      --> Resolusi Nama Kota menjadi Lintang & Bujur (lat, lon)                |
|                                                                               |
|   2. Open-Meteo Forecast API:   /v1/forecast?latitude={lat}&longitude={lon}   |
|      --> Pengambilan Suhu, Kelembapan, Kode WMO, Kecepatan Angin              |
|                                                                               |
|   3. OpenWeatherMap CDN:        /img/wn/{icon}@4x.png                         |
|      --> Aset Visual Grafis Ikon Cuaca Beresolusi Tinggi                      |
+-------------------------------------------------------------------------------+
```

2.2 Arsitektur Sistem Modular Berbasis Lapisan (*Layered Architecture*)

Struktur program dalam `script.js` dibangun menggunakan prinsip pemisahan tanggung jawab (*Separation of Concerns*):

1. **API Service Layer (`WeatherService`)**:
    - Bertindak murni sebagai jembatan komunikasi jaringan HTTP.
    - Tidak memiliki keterkaitan langsung dengan DOM ataupun elemen visual.
    - Mengambil parameter input murni (*primitive types*), mengeksekusi request jaringan, memvalidasi integritas respons JSON, memetakan kode cuaca standar WMO (*World Meteorological Organization*), dan mengembalikan objek terstruktur.
2. **UI Controller Layer (`UIController`)**:
    - Berperan sebagai pengelola antarmuka pengguna dan orkestrator interaksi.
    - Mengelola cache elemen DOM untuk menghindari pemanggilan `document.getElementById` berulang yang tidak efisien.
    - Mengatur siklus visual: menampilkan status *loading*, memformat waktu lokal (`Intl.DateTimeFormat`), menyaring masukan teks (*input trimming*), dan memperbarui elemen antarmuka saat data sukses diterima.

2.3 Alur Kerja Integrasi Multi-Endpoint REST API (*Chained Asynchronous Flow*)

Karena Open-Meteo memisahkan fungsionalitas geolokasi dan ramalan cuaca demi efisiensi spasial, aplikasi mengeksekusi alur data asinkron berantai:

1. **Pengguna Mengirimkan Form**: Pengguna mengetik nama kota (misal: `"Balige"`) dan menekan tombol *Cari Cuaca*.
2. **Validasi Klien & Loading State**: `handleSearch()` memastikan teks tidak kosong, mengaktifkan spinner loading, dan menonaktifkan tombol submit.
3. **Tahap 1 (Geocoding)**: Memanggil `WeatherService.getCityCoordinates(city)` . API merespons dengan nilai `latitude` , `longitude` , nama resmi, dan kode negara.

4. **Tahap 2 (Meteorological Fetch)**: Koordinat yang diperoleh diteruskan langsung ke `WeatherService.getCurrentWeather(latitude, longitude)`. API merespons dengan data `temperature_2m`, `relative_humidity_2m`, `weather_code`, dan `wind_speed_10m`.
5. **Tahap 3 (Data Transformation)**: `weather_code` numerik (misal: `61`) dipetakan via `mapWeatherCode()` menjadi deskripsi berbahasa Indonesia (`"Hujan Ringan"`) dan ikon URL OpenWeatherMap (`"10d"`).
6. **Tahap 4 (DOM Presentation)**: `renderWeather()` memperbarui nilai pada elemen HTML semantik dan memunculkan kartu cuaca dengan animasi transisi *fade-in*.
7. **Pembersihan Akhir**: Blok `finally` memastikan status loading dinonaktifkan terlepas dari berhasil atau gagalnya request.

2.4 Spesifikasi Kebutuhan Fungsional dan Non-Fungsional

A. Kebutuhan Fungsional:

- **[F-01] Pencarian Cuaca Lintas Wilayah**: Pengguna dapat memasukkan nama kota lokal maupun internasional untuk memperoleh kondisi cuaca terkini.
- **[F-02] Penampilan Parameter Cuaca Lengkap**: Sistem menyajikan data suhu aktual (°C), deskripsi cuaca, ikon visual, kelembapan udara (%), dan kecepatan angin (m/s).
- **[F-03] Pemuatan Kota Standar (Default City Auto-Load)**: Sistem secara otomatis menampilkan cuaca kota Jakarta saat aplikasi pertama kali dimuat tanpa menunggu aksi pengguna.
- **[F-04] Penanganan Kesalahan Terpadu**: Menampilkan pesan galat spesifik apabila kota tidak ditemukan (HTTP 404/Empty result) atau terjadi kegagalan jaringan (*network offline*).
- **[F-05] Format Waktu Lokal Terstandarisasi**: Menampilkan stempel waktu pembaruan data dalam format bahasa Indonesia (`id-ID`).

B. Kebutuhan Non-Fungsional:

- **[NF-01] Performa Cepat (*Zero Dependency*)**: Tidak memuat pustaka eksternal berukuran besar; ukuran total bundel berkas di bawah 50 KB.
- **[NF-02] Aksesibilitas (A11y)**: Memiliki kontras warna tinggi, label formulir untuk *screen readers*, dan atribut ARIA *live region*.
- **[NF-03] Desain Responsif**: Antarmuka adaptif terhadap layar ponsel cerdas (lebar 320px–480px) hingga layar desktop resolusi tinggi.

BAB III: PROSES INTERAKSI DAN REKAYASA PROMPT DENGAN AI GENERATIF

3.1 Skenario Rekayasa Kebutuhan dan Prompt Engineering

Dalam proses pengembangan aplikasi ini, mahasiswa bertindak sebagai arsitek perangkat lunak utama dan auditor kode, sementara Generative AI (Google Gemini / Claude) diposisikan sebagai asisten teknis pelaksana. Agar keluaran AI terstruktur dan mematuhi batasan *pure vanilla*, mahasiswa menggunakan metode **Multi-Turn Role-Based Prompting** yang membagi proses ke dalam tiga iterasi bertahap.

3.2 Iterasi Prompt I: Perancangan Semantik HTML5 Aksesibel dan Sistem Desain Glassmorphism CSS3

A. Teks Instruksi (Prompt) kepada AI:

> ***Instruksi Mahasiswa:*** **"Bertindaklah sebagai Senior Frontend Web Developer dan UI/UX Designer. Saya ingin membangun antarmuka web untuk aplikasi cuaca 'Weather Web App' menggunakan HTML5 murni dan CSS3 murni tanpa framework apa pun. Buatkan struktur berka* `index.html` *dan* `style.css` *dengan kriteria:*
>
> 1. *HTML5 harus semantik (*
>
> *,*
>
> *,*
>
> *,*
>
> *,*
>
> *,*
>
> *) dan mematuhi prinsip aksesibilitas WAI-ARIA (aria-live pada status feedback, visually-hidden label, icon svg).*
> 2. *Antarmuka memiliki form pencarian, indikator loading spinner CSS, kotak alert error yang ramah pengguna, dan kartu cuaca utama dengan metrik sekunder (kelembapan dan kecepatan angin dalam CSS Grid 2 kolom).*
> 3. *Terapkan estetika modern: Glassmorphism (backdrop-filter: blur, border tipis semi-transparan), CSS Custom Properties untuk sistem warna gelap yang elegan, Google Fonts 'Plus Jakarta Sans', dan tata letak responsif Mobile-First."**

B. Rasionalisasi Pemilihan Prompt:

Prompt ini menetapkan batasan ketat sejak awal: melarang ketergantungan pada Tailwind/Bootstrap demi mempertahankan kendali kode murni, menegaskan standar aksesibilitas agar elemen form tidak membingungkan pengguna disabilitas, serta mendikte penggunaan *CSS Custom Properties* agar palet warna mudah dipelihara dan konsisten.

3.3 Iterasi Prompt II: Pembangunan Lapisan Layanan API (WeatherService) dengan Chained Fetch *A.*

Teks Instruksi (Prompt) kepada AI:

> ***Instruksi Mahasiswa:** *"Sekarang fokus pada logika integrasi data cuaca di `script.js` . Buatkan objek `WeatherService` menggunakan JavaScript ES6+ modern. Objek ini harus menangani dua pemanggilan API publik Open-Meteo secara berantai (chained async):*
>
> 1. *`getCityCoordinates(city)` : Memanggil Open-Meteo Geocoding API untuk mengonversi nama kota menjadi koordinat lintang dan bujur. Tangani validasi jika kota tidak ditemukan.*
> 2. *`getCurrentWeather(latitude, longitude)` : Memanggil Open-Meteo Forecast API untuk mengambil data suhu (temperature_2m), kelembapan (relative_humidity_2m), kode cuaca (weather_code), dan kecepatan angin (wind_speed_10m) dengan satuan m/s.*
> 3. *`mapWeatherCode(code)` : Fungsi untuk menerjemahkan kode cuaca standar WMO menjadi teks bahasa Indonesia yang ramah pengguna dan tautan gambar ikon OpenWeatherMap."**

B. Rasionalisasi Pemilihan Prompt:

Mahasiswa secara terarah memisahkan instruksi lapisan data dari antarmuka agar AI tidak menyatukan kode manipulasi DOM ke dalam fungsi jaringan HTTP. Hal ini menjamin penerapan prinsip *Single Responsibility Principle* (SRP) pada modul JavaScript.

3.4 Iterasi Prompt III: Orkestrasi Lapisan Antarmuka (UIController), Penanganan Asinkron, dan Pemetaan WMO

A. Teks Instruksi (Prompt) kepada AI:

> ***Instruksi Mahasiswa:** *"Buatkan lapisan kedua pada `script.js` yaitu objek `UIController`. Objek ini bertugas:*
>
> 1. *Menyimpan cache referensi seluruh elemen DOM di awal agar performa tinggi.*
> 2. *Mengatur fungsi state tampilan: `showLoading()`, `hideLoading()`, `showError(message)`, dan `renderWeather(data)`.*
> 3. *Membuat fungsi `handleSearch(city)` yang menghubungkan input pengguna dengan `WeatherService`. Terapkan penanganan siklus try-catch-finally agar status loading selalu ditutup dengan benar.*
> 4. *Menginisialisasi event listener submit form (mencegah reload halaman via e.preventDefault()) dan otomatis memanggil cuaca kota 'Jakarta' saat event DOMContentLoaded terjadi."**

B. Rasionalisasi Pemilihan Prompt:

Alasan pemberian prompt ini adalah untuk memandu AI menyusun logika orkestrasi asinkron yang aman (*robust async lifecycle*), mencegah *memory leak* dari query DOM berulang, dan memberikan pengalaman pengguna yang responsif (*instant feedback*) sejak aplikasi dibuka.

BAB IV: EVALUASI KRITIS DAN VERIFIKASI TEKNIS MANDIRI

Meskipun Generative AI mampu menghasilkan kerangka kode dasar secara instan, keluaran mentah (*raw output*) dari model mengandung sejumlah kelemahan substansial dan cacat arsitektur yang jika tidak ditinjau oleh mahasiswa, akan menyebabkan kegagalan sistem pada kondisi nyata (*production breakdown*). Mahasiswa melakukan audit mendalam dan menemukan empat masalah teknis utama.

4.1 Identifikasi Anomali dan Cacat Logika pada Kode Bawaan AI

Berikut adalah ringkasan anomali teknis yang ditemukan pada kode awal keluaran AI:

No	Modul / Komponen	Anomali Kode Awal AI	Dampak Risiko	Tindakan Refactoring Mahasiswa
1	`WeatherService.getCityCoordinates`	Mengabaikan pemeriksaan `response.ok` pada `fetch()`	*Unhandled Silent Error* saat HTTP 404/500	Menambahkan pengecekan eksplisit `if (!response.ok)`
2	`UIController.renderWeather`	Penggunaan `element.innerHTML` untuk data teks	Kerentanan *Cross-Site Scripting* (XSS)	Mengganti ke `textContent` dan sanitasi input
3	`UIController.handleSearch`	Ketiadaan blok `finally` pada struktur asinkron	Indikator *loading spinner* macet selamanya	Mengenkapsulasi `hideLoading()` di dalam blok `finally`
4	`WeatherService.mapWeatherCode`	Pemetaan kode cuaca WMO sangat minim (<5 kode)	Tampilan cuaca *crash* saat cuaca berkabut/badai	Menyusun *dictionary* komprehensif 27 kode WMO lengkap

---

4.2 Analisis Masalah I: Karakteristik Fetch API (Ketiadaan Reject Otomatis pada HTTP 404/500)

A. Penjelasan Teknis Masalah:

Pada fungsi awal yang dihasilkan AI, pemanggilan REST API ditulis sebagai berikut:

```
// KODE AWAL DARI AI (CACAT LOGIKA JARINGAN) async getCityCoordinates(city) {     const res = await fetch(`https://geocoding-api.open-meteo.com/v1/search?name=${cit y}`);     const data = await res.json();         return data.results[0]; // Crash: TypeError jika results bernilai undefined! }
```

Spesifikasi *Fetch API* standar W3C menyatakan bahwa `fetch()` **hanya akan melakukan reject Promise jika terjadi kegagalan jaringan fisik (misal: kabel terputus atau DNS lookup failure)**. Jika server

merespons dengan status HTTP `404 Not Found` atau `500 Internal Server Error` , Promise tetap berstatus *resolved*. Karena kode AI langsung mengeksekusi `data.results[0]` , aplikasi mengalami *Uncaught TypeError: Cannot read properties of undefined* yang menghentikan eksekusi skrip tanpa pesan kesalahan yang jelas kepada pengguna.

B. Solusi Refactoring Mandiri Mahasiswa:

Mahasiswa menambahkan validasi bertingkat:

1. Mengamankan parameter query menggunakan `encodeURIComponent(city)` untuk mencegah kerusakan URL akibat spasi atau karakter khusus.
2. Memeriksa properti `response.ok` (status HTTP 200–299). Jika `false` , sistem melontarkan `Error` eksplisit dengan kode status server.
3. Memverifikasi ketersediaan larik `data.results` . Jika kosong, sistem melontarkan galat bersahabat: *"Kota tidak ditemukan. Periksa kembali ejaan nama kota Anda."*

4.3 Analisis Masalah II: Kerentanan Keamanan XSS (*Cross-Site Scripting*) pada Manipulasi DOM

A. Penjelasan Teknis Masalah:

AI awalnya menyarankan pembaruan DOM menggunakan manipulasi string HTML mentah:

```
// KODE AWAL DARI AI (RENTAN TERHADAP INJEKSI XSS)
cityNameElement.innerHTML = `<span>${location.name}, ${location.country}</span>`;
```

Jika data respons geocoding disusupi atau input pengguna diteruskan tanpa sanitasi ke elemen DOM melalui `innerHTML` , peramban akan mem-parsing string tersebut sebagai elemen HTML/JavaScript aktif. Ini membuka celah serangan injeksi *Cross-Site Scripting* (DOM-based XSS).

B. Solusi Refactoring Mandiri Mahasiswa:

Mahasiswa menghapus seluruh penggunaan `innerHTML` untuk penulisan konten dinamis dan menggantinya secara konsisten dengan properti aman: `this.elements.cityName.textContent = `${location.name}, ${location.country}`;`

Penggunaan `textContent` memaksa peramban memperlakukan input murni sebagai data tekstual (*string literal*), sehingga skrip berbahaya secara otomatis dinetralisir (*escaped*). Selain itu, mahasiswa menambahkan metode `city.trim()` di awal fungsi pencarian untuk mengeliminasi spasi kosong yang tidak disengaja.

4.4 Analisis Masalah III: Jebakan Asinkron (*Stuck Loading State* Akibat Ketiadaan Blok Finally)

A. Penjelasan Teknis Masalah:

Pada alur pencarian `handleSearch()`, AI menonaktifkan spinner loading di akhir blok `try`:

```
// KODE AWAL DARI AI (RENTAN MACET)
async handleSearch(city) {
    this.showLoading();
    try {
        const loc = await WeatherService.getCityCoordinates(city);
        const weather = await WeatherService.getCurrentWeather(loc.latitude, loc.longit
ude);
        this.renderWeather(weather);
        this.hideLoading(); // HANYA DIJALANKAN JIKA TIDAK ADA ERROR
    } catch (err) {
        this.showError(err.message);
        // Lupa memanggil hideLoading()!
    }
}
```

Ketika pengguna memasukkan nama kota fiktif atau terjadi pemutusan koneksi internet saat request tahap kedua, alur program melompat langsung ke blok `catch`. Akibatnya, `this.hideLoading()` tidak pernah dieksekusi. Tombol pencarian tetap dalam kondisi *disabled* (`"Mencari..."`) dan spinner loading terus berputar tanpa akhir di atas pesan error, merusak integritas pengalaman pengguna (*UX deadlock*).

B. Solusi Refactoring Mandiri Mahasiswa:

Mahasiswa menerapkan blok `finally` yang secara deterministik pasti dieksekusi oleh mesin JavaScript peramban, baik saat request sukses maupun saat terinterupsi oleh galat:

```
try {
    // Eksekusi pengambilan dan rendering data...
} catch (error) {
    this.showError(userFriendlyMessage);
} finally {
    // Menjamin kontrol form selalu diaktifkan kembali
    this.hideLoading();
}
```

4.5 Analisis Masalah IV: Ketidaklengkapan Pemetaan Standar Meteorologi (WMO Weather Codes)

AI pada awalnya hanya mendefinisikan 4 kode cuaca (0 = Cerah, 1 = Berawan, 2 = Hujan, 3 = Badai). Padahal standar meteorologi internasional WMO pada Open-Meteo memuat puluhan variasi kondisi, seperti kabut tebal (`45`), gerimis bertingkat (`51-57`), hujan salju (`71-77`), hingga badai petir disertai es (`99`). Ketiadaan kode ini menyebabkan aplikasi menampilkan deskripsi `undefined` atau ikon kosong (*broken image*).

Mahasiswa menyusun kamus data komprehensif 27 kode WMO lengkap dengan translasi deskripsi bahasa Indonesia yang akurat dan pemetaan ikon OpenWeatherMap beresolusi tinggi (`@4x.png`).

4.6 Perbandingan Kode Komparatif: Sebelum (*AI-Generated*) vs. Sesudah Refactoring (*Human-Verified*)

Berikut adalah perbandingan*code diff* aktual pada berkas `script.js` yang membuktikan intervensi dan audit kritis mahasiswa:

Perbandingan Refactoring Kode (Human Audit vs. Raw AI)

```
 const WeatherService = {
   async getCityCoordinates(city) {
-     // Cacat AI: Rentan terhadap URL Malformed, ketiadaan validasi status HTTP
-     const endpoint = `https://geocoding-api.open-meteo.com/v1/search?name=${city}`;
-     const response = await fetch(endpoint);


-     const data = await response.json();
-     return data.results[0];
+     // Solusi Mahasiswa: encodeURIComponent, validasi HTTP ok, dan simulasi 404 ramah
+    const endpoint = `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(ci
ty)}&count=1&language=id&format=json`; +     const response = await fetch(endpoint);
+
+     if (!response.ok) {
+       throw new Error(`Gagal menghubungi server pencarian lokasi (Status: ${response.status}
${re sponse.statusText}).`); +     }
+ +     const data = await
response.json();
+
+     if (!data.results || data.results.length === 0) { +       throw new Error(&amp;quot;Kota
tidak ditemukan. Periksa kembali ejaan nama kota Anda.&amp;quot;);
+     }
+ +     const [firstResult] =
data.results;
+     return {
+       latitude: firstResult.latitude,
+       longitude: firstResult.longitude,
+       name: firstResult.name, +       country:
firstResult.country || &amp;#39;Internasional&amp;#39;
+     };


   },     async
handleSearch(city) {

-       // Cacat AI: Tidak ada sanitasi spasi, ketiadaan blok finally menyebabkan spinner macet-
this.showLoading();
-       try {
-       const loc = await WeatherService.getCityCoordinates(city);-       const weather = await
WeatherService.getCurrentWeather(loc.latitude, loc.longitude);
-       this.renderWeather(weather);
-       this.hideLoading();
-       } catch (err) { -       this.showError(err.message);
-       }
```

```
+    // Solusi Mahasiswa: Trim sanitasi, proteksi error jaringan offline, dan finally block
+    const sanitizedCity = city.trim();
+    if (!sanitizedCity) { +      this.showError(&#39;Harap masukkan nama
kota terlebih dahulu.&#39;);
+      this.elements.cityInput.focus();
+      return;
+    }
+ +
this.showLoading();
+    try {
+      const location = await WeatherService.getCityCoordinates(sanitizedCity);
+     const weather = await WeatherService.getCurrentWeather(location.latitude, location.longitud
e); +      const weatherInfo = WeatherService.mapWeatherCode(weather.weatherCode);
+
+      this.renderWeather({
+        cityName: `${location.name}, ${location.country}`,
+        temperature: weather.temperature,
+        humidity: weather.humidity,
+        windSpeed: weather.windSpeed,
```

```
+        description: weatherInfo.description,
+        iconUrl: weatherInfo.iconUrl
+      });
+    } catch (error) {
+      console.error(&#39;Error saat mengambil data cuaca:&#39;, error);
+      let userFriendlyMessage = error.message;
+      if (error instanceof TypeError && error.message.toLowerCase().includes(&#39;fetch&#39;)) {
+        userFriendlyMessage = &#39;Koneksi internet bermasalah atau server tidak dapat dijangkau.
Periksa kembali koneksi Anda.&#39;;
+      }
+      this.showError(userFriendlyMessage);
+    } finally {
+      this.hideLoading(); // Menjamin spinner loading selalu ditutup dengan aman
+    }
  }
};
```

BAB V: STRATEGI PENGELOLAAN REPOSITORI DAN KONTROL VERSI

5.1 Struktur Repositori Terorganisir dan Standar Berkas Proyek

Proyek aplikasi web ini dikelola secara terpusat pada platform kendali versi **GitHub** dengan struktur berkas yang bersih, transparan, dan mudah dipahami:

```
Weather APPS/
├── .git/                       # Basis data histori version control Git
├── .github/                    # Konfigurasi repositori dan alur kerja CI
├── .gitignore                  # Filter pengecualian file sampah sistem (macOS DS_Stor
e, logs)
├── index.html                  # Struktur dokumen semantik HTML5 & modul antarmuka
├── style.css                  # Sistem desain, variabel warna, CSS Grid, Glassmorphism
├── script.js                   # Logika bisnis (WeatherService & UIController)
├── README.md                   # Dokumentasi teknis proyek, fitur, dan live demo
└── PROPOSAL_TUGAS_1_PABWE.pdf # Berkas PDF terformat untuk penilaian akademik
```

5.2 Standar Konvensi Commit (*Conventional Commits 1.0.0*)

Setiap perubahan kode dicatat secara granular menggunakan standar **Conventional Commits** untuk mencerminkan riwayat rekayasa yang profesional dan dapat diaudit:

- `feat(ui): implement accessible semantic html5 structure and glassmorphism styling`
- `feat(api): integrate open-meteo geocoding and forecast rest api endpoints`
- `feat(wmo): map full 27 wmo meteorological weather codes with id descriptions`
- `fix(network): handle fetch api non-200 responses and missing results explicitly`
- `fix(ux): add finally block to prevent stuck loading spinner on network failure`
- `refactor(dom): eliminate innerhtml usage with secure textcontent to prevent xss`
- `docs(proposal): complete comprehensive academic proposal draft for assignment 1`

5.3 Kebijakan Transparansi, Lisensi, dan Aksesibilitas Dosen/Asisten

Mahasiswa memberikan komitmen penuh terhadap transparansi akademik dengan menetapkan:

- **Tautan Repositori GitHub Publik**: `https://github.com/yogitm/weather-web-app`
- **Tautan Live Demo Aplikasi**: `https://weather.yogitrim.my.id`
- **Aksesibilitas Pengujian Langsung**: Repositori berstatus **Publik (*Public*)** sehingga Dosen Pengampu mata kuliah PABWE dan seluruh Tim Asisten Dosen dapat mengkloning dan meninjau kode sumber kapan pun tanpa batasan izin tertutup.
- **Kemudahan Replikasi Lingkungan (*Zero Config*)**: Karena aplikasi berbasis *Pure Vanilla Web*, penguji tidak perlu menginstal dependensi *node_modules* yang berat. Aplikasi dapat langsung diuji cukup dengan membuka berkas `index.html` pada peramban web modern (Google Chrome / Mozilla Firefox / Safari / Microsoft Edge) atau melalui ekstensi *Live Server*.

BAB VI: RENCANA IMPLEMENTASI DAN LINIMASA KERJA (1 MINGGU)

Pelaksanaan Tugas I dirancang ke dalam jadwal kerja terstruktur selama 7 hari kalender:

Hari	Tahapan / Milestone	Aktivitas Teknis yang Dilaksanakan	Luaran (*Deliverables*)
Hari ke-1	Perencanaan & Analisis REST API	- Riset dokumentasi publik Open-Meteo API. - Pemetaan parameter koordinat dan format data cuaca WMO. - Perancangan sketsa wireframe tata letak antarmuka.	Skema integrasi API & daftar variabel data.
Hari ke-2	Pembangunan UI & Desain Sistem	- Penulisan struktur semantik `index.html` berbasis A11y. - Pembuatan `style.css` dengan CSS Variables dan efek Glassmorphism.	Tampilan statis antarmuka aplikasi.
Hari ke-3	Implementasi Lapisan Jaringan (API)	- Pembuatan modul `WeatherService` pada `script.js`. - Pengujian endpoint Geocoding & Forecast via `fetch()` asinkron.	Fungsi request data cuaca berfungsi via console.
Hari ke-4	Implementasi UI Controller & DOM Logic	- Pembuatan modul `UIController` dan integrasi event listener. - Implementasi pemetaan 27 kode cuaca WMO danauto-load Jakarta.	Aplikasi berfungsi secara end-to-end.
Hari ke-5	Audit Kritis, Security Review, & Refactoring	- Audit kode keluaran AI: mitigasi XSS, pencegahan *stuck loading spinner*, dan penambahan validasi status HTTP non-200.	Kode yang tangguh terhadap kesalahan (*faulttolerant*).
Hari ke-6	Pengujian Lintas Perangkat & Responsivitas	- Pengujian tampilan pada viewport mobile (320px–480px) & desktop. - Pengujian kondisi ekstrem (kota salah eja, modeoffline browser).	Verifikasi kualitas antarmuka & fungsionalitas.
Hari ke-7	Finalisasi Dokumen Proposal & Publikasi Git	- Penyusunan dokumen proposal lengkap dan kompilasiPDF. - Sinkronisasi berkas repositori GitHub dan publikasilive demo.	Dokumen proposal final & repositori siap dinilai.

BAB VII: KESIMPULAN DAN REFLEKSI KRITIS MAHASISWA

Pengembangan aplikasi **Weather Web App** untuk memenuhi Tugas I mata kuliah **11S3133 Pengembangan Aplikasi Berbasis Web** telah membuktikan bahwa pemanfaatan *Generative Artificial Intelligence* memberikan akselerasi yang luar biasa dalam siklus rekayasa perangkat lunak web, khususnya dalam menghasilkan struktur dasar dokumen semantik, sistem variabel warna CSS modern, dan kerangka fungsi asinkron.

Akan tetapi, pengalaman empiris dalam pengerjaan proyek ini menghasilkan satu refleksi kritis yang fundamental: **Generative AI hanyalah sebuah mesin pembuat purwarupa sintaksis, bukan arsitek keandalan perangkat lunak.**

Kelemahan-kelemahan kritis yang ditemukan pada kode bawaan AI—seperti ketidakmampuan menangani respon galat `fetch()` secara mandiri, celah keamanan manipulasi DOM yang membuka peluang XSS, hingga ketiadaan blok `finally` yang menyebabkan indikator *loading* macet selamanya—membuktikan bahwa otomasi AI tanpa supervisi manusia yang kompeten adalah tindakan yang berbahaya bagi keamanan sistem informasi.

Sebagai mahasiswa program studi S-1 Informatika di Institut Teknologi Del, integritas akademik dan tanggung jawab profesional pengembang diwujudkan melalui kemampuan berpikir kritis (*critical thinking*), penguasaan mendalam atas konsep komputasi dasar, serta verifikasi teknis independen terhadap setiap baris kode yang dihasilkan. Pada akhirnya, keberhasilan sebuah aplikasi web dinilai bukan dari seberapa banyak kecerdasan buatan yang digunakan, melainkan dari seberapa aman, andal, dan bermanfaat aplikasi tersebut bagi pengguna akhir.

DAFTAR PUSTAKA

1. Mozilla Developer Network (MDN). (2024). *Using the Fetch API & Asynchronous JavaScript*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
2. World Wide Web Consortium (W3C). (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. https://www.w3.org/TR/WCAG22/
3. Open-Meteo. (2024). *Open-Meteo Weather Forecast & Geocoding API Documentation*. https://openmeteo.com/en/docs
4. Open Web Application Security Project (OWASP). (2023). *DOM-based Cross-Site Scripting (XSS) Prevention Cheat Sheet*. OWASP Foundation. https://cheatsheetseries.owasp.org/
5. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
6. World Meteorological Organization (WMO). (2021). *Manual on Codes: International Codes, Volume I.1 (WMO-No. 306)*. Secretariat of the World Meteorological Organization.